TEMCO LEARN

T3000 API & Protocol

Developer Tools API · built Nov 13, 2026

Developer Tools API

Overview

The Developer Tools API provides debugging and inspection capabilities including file browsing, database queries, and system logs.

Common Operations

Browse Files

TYPESCRIPT

```typescript
const files = await fetch('http://localhost:9103/api/develop/files/list?path=/')
  .then(res => res.json());

console.log(files);
// [
//   { name: "database.db", type: "file", size: 52428800 },
//   { name: "logs", type: "directory" }
// ]
```

Query Database

TYPESCRIPT

```typescript
const result = await fetch('http://localhost:9103/api/develop/database/query', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    sql: 'SELECT * FROM DEVICES LIMIT 10'
  })
}).then(res => res.json());
```

Get Logs

```typescript
const logs = await fetch('http://localhost:9103/api/develop/logs/get?lines=100')
  .then(res => res.json());
```

File Browser Endpoints

GET /api/develop/files/list

Description: List files and directories

Parameters:

- `path` (query, string) - Directory path

Request:

```http
GET /api/develop/files/list?path=/Database HTTP/1.1
```

Response:

```json
{
  "path": "/Database",
  "items": [
    {
      "name": "t3000.db",
      "type": "file",
      "size": 52428800,
      "modified": "2026-01-09T08:00:00Z"
    },
    {
      "name": "backups",
      "type": "directory"
    }
  ]
}
```

Used By: FileBrowser.tsx

Handler: `api/src/t3_develop/file_browser/routes.rs::list_files`

GET /api/develop/files/read

Description: Read file contents

Parameters:

- `path` (query, string) - File path

Request:

```HTTP
GET /api/develop/files/read?path=/logs/app.log HTTP/1.1
```

Response:

```TEXT
2026-01-09 08:00:00 INFO Server started on port 9103
2026-01-09 08:00:05 INFO Device 1 connected
```

Used By: FileViewer.tsx

Database Viewer Endpoints

GET /api/develop/database/list

Description: List all database files

Request:

```HTTP
GET /api/develop/database/list HTTP/1.1
```

Response:

```JSON
[
  {
    "name": "t3000.db",
    "path": "/Database/t3000.db",
    "size": 52428800,
    "tables": 45
  }
]
```

Used By: DatabaseViewer.tsx

Handler: `api/src/t3_develop/database_viewer/routes.rs::list_databases`

POST /api/develop/database/query

Description: Execute SQL query

Request:

```HTTP
POST /api/develop/database/query HTTP/1.1
Content-Type: application/json

{
  "sql": "SELECT id, serial_number, product_name FROM DEVICES LIMIT 10",
  "database": "t3000.db"
}
```

Response:

JSON

```
{
  "columns": ["id", "serial_number", "product_name"],
  "rows": [
    [1, 123456, "T3-BB"],
    [2, 654321, "T3-LB"]
  ],
  "row_count": 2,
  "execution_time_ms": 5
}
```

Used By: QueryEditor.tsx

Handler: `api/src/t3_develop/database_viewer/routes.rs::execute_query`

Errors:

- `400` - Invalid SQL syntax
- `403` - Forbidden operation (DROP, DELETE without WHERE, etc.)

GET /api/develop/database/schema

Description: Get database schema information

Parameters:

- `database` (query, string) - Database name

Request:

HTTP

```
GET /api/develop/database/schema?database=t3000.db HTTP/1.1
```

Response:

JSON

```
{
  "tables": [
    {
      "name": "DEVICES",
      "columns": [
        { "name": "id", "type": "INTEGER", "primary_key": true },
        { "name": "serial_number", "type": "INTEGER", "nullable": false }
      ],
      "indexes": ["idx_devices_serial"]
    }
  ]
}
```

Used By: SchemaViewer.tsx

System Logs Endpoints

GET /api/develop/logs/get

Description: Get application logs

Parameters:

- `lines` (query, integer, optional) - Number of lines (default: 100)
- `level` (query, string, optional) - Filter by level (INFO, WARN, ERROR)

Request:

HTTP

```
GET /api/develop/logs/get?lines=50&level=ERROR HTTP/1.1
```

Response:

JSON

```
{
  "lines": [
    {
      "timestamp": "2026-01-09T08:00:00Z",
      "level": "ERROR",
      "message": "Failed to connect to device 192.168.1.100"
    }
  ],
  "total": 50
}
```

Used By: LogViewer.tsx

Handler: `api/src/t3_develop/system_logs/routes.rs::get_logs`

Implementation Details

File Browser Security

File browser is restricted to specific directories:

- `/Database` - Database files
- `/logs` - Log files
- `/uploads` - User uploads

Paths outside these directories are blocked.

Query Safety

SQL queries are validated to prevent:

- DROP, DELETE, UPDATE without WHERE clause
- ATTACH DATABASE
- System table modifications

TypeScript Types

TYPESCRIPT

```typescript
export interface FileItem {
  name: string;
  type: 'file' | 'directory';
  size?: number;
  modified?: string;
}

export interface QueryResult {
  columns: string[];
  rows: any[][];
  row_count: number;
  execution_time_ms: number;
}

export interface LogEntry {
  timestamp: string;
  level: 'INFO' | 'WARN' | 'ERROR' | 'DEBUG';
  message: string;
}
```

Error Codes

- `FORBIDDEN_PATH` - Access to path denied
- `FILE_NOT_FOUND` - File doesn't exist
- `INVALID_SQL` - SQL syntax error
- `UNSAFE_QUERY` - Query blocked by safety checks